

Dear examiner,

please find the criteria marks as well as teacher's comments below.

A: 4	B: 3	C: 3	D: 2	E: 5	Total: 17
------	------	------	------	------	-----------

Criterion A: Presentation

A very well-written and easy to follow IA. The introduction has a clear aim, which is fully met. The exploration's structure is well-developed (introduction, body, conclusion); all sections link well to each other and the reader never feels lost. Equations, diagrams and tables are used effectively to aid understanding. There are no irrelevant or repetitive calculations.

Criterion B: Mathematical communication

The student uses relevant, appropriate mathematical communication. Equations and many insightful diagrams are used effectively to aid understanding. All key terms and variables are defined.

Criterion C: Personal engagement

The student demonstrates outstanding personal engagement. He took a textbook problem and extended it using very creative and original approaches, presenting the concepts in a unique way. The IA touches on two-variable optimization, including partial derivatives, which is unfamiliar mathematics the student taught to himself and explained in a way that a fellow student can comprehend. All diagrams are his own work; the student implemented his own code in Python.

Criterion D: Reflection

Meaningful reflection is seen throughout the IA, in particular on pages 4, 5, 15, as well as at the end. The student successfully keeps linking back to the exploration's aim. In his conclusion the student discusses possible extensions to his work. However, there should have been more emphasis on discussing limitations to his modeling and most of the reflection seen is not critical.

Criterion E: Use of mathematics

The student uses relevant mathematics commensurate with the level of the course. All mathematical arguments are clear and logical; all mathematics explored is error-free and an appropriate degree of accuracy is used at all times. Thorough knowledge and understanding are demonstrated as the student supports all his mathematical working by clear and logical written explanations and diagrams. The student successfully applies the mathematics and really plays with it. However, the explanations in section 3.2 on two-variable optimization should have been in more detail to achieve the top score in this criterion.

How the Application of Mathematical Optimization can be used to extend the Lifeguard Problem

hdv587

1 Introduction

In a press release from the 09.03.2021 the “DLRG Deutsche Lebens-Rettungs-Gesellschaft e.V.” (German Association for Lifeguarding) announced that in 2020, 378 people died by drowning in German rivers, lakes and at the coast (DLRG). In 2019 Lifeguards of the DLRG were able to save 950 lives (DLRG).

Therefore, lifeguards are positioned at popular beaches to save swimmers in peril. If they observe an emergency, they hurry into the water to help the person in trouble. As any second counts, it is desirable to find an optimal way to reach the person as fast as possible.



Figure 1: Lifeguard post at the beach of La-Croix Valmer (Tourisme)

The aim of this IA is to introduce and optimize three different scenarios that extend on the quite well known and solved lifeguard optimization. The base problem will be explained and optimized. From there, new concepts about optimization will be introduced that will be used to extend the beach problem to suggest optimal paths for a lifeguard to save a swimmer in need. This problem is relevant to be solved because the solutions can be generalized and used to train the lifeguard’s intuition to allow them to do their job more efficiently.

2 Problem

2.1 Optimal path problem

Consider a beach with a straight waterfront. A lifeguard in his watch post sees a swimmer in danger in the water. He wants to reach the swimmer as quickly as possible. Intuitively, the best option would be to simply take the direct path to the swimmer in need, crossing the boundary between beach and water on the line between the watch post and swimmer. The issue is that the lifeguard is able to move faster on the beach than in the water. This makes it inefficient to swim such a large portion of the path. With this specific situation, there is an optimum where the lifeguard runs most of the way on the beach and swims the rest. The optimum can be defined in terms of the location of where the lifeguard crosses the boundary between beach and water. Using this parameter, the distance the lifeguard has to run as well as the distance he has to swim can be calculated. Given the speed of the lifeguard for each medium, it is possible to determine the time the lifeguard takes for each section, which can then be added to find the total time to reach the swimmer in need. Using optimization, the value of the parameter that minimizes the time can be determined.

The following is the base case of the lifeguard problem.

The lifeguard post is located at $(0,0)$ of this coordinate system. The swimmer is located at $(150,150)$. The lifeguard post is 50 away from the waterfront. The goal is now to formulate two equations that correspond to the two distances from the lifeguard post to the waterfront and from the waterfront to the swimmer.

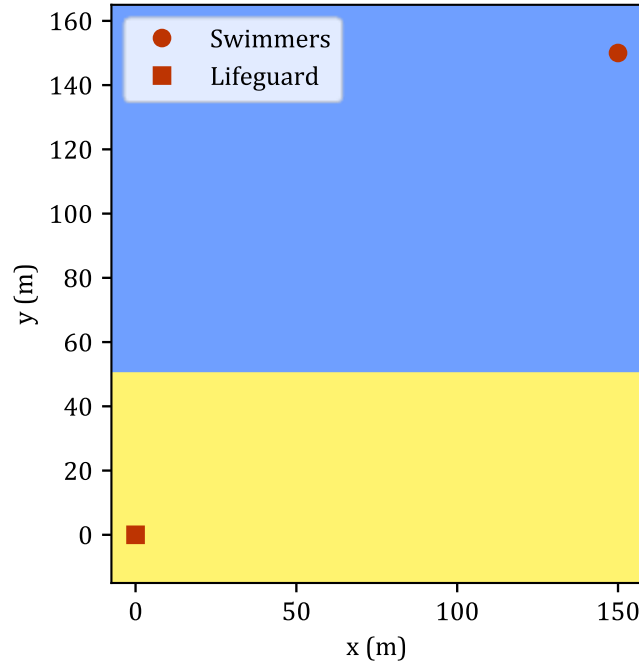


Figure 2: The standard beach situation (own work)

Say there is a point P on the line of the waterfront. Its y coordinate is given because the line has a gradient of zero. In this case the y -value is equal to 50. Its x component x_p is variable.

The distance from the lifeguard post to point P is can be calculated using Pythagoras.

$$d_1 = \sqrt{50^2 + x_p^2}$$

If the distance to the waterfront is expressed as W ,

$$d_1 = \sqrt{W^2 + x_p^2}$$

The distance from point P to the swimmer is also determined using Pythagoras.

$$d_2 = \sqrt{(150 - 50)^2 + (150 - x_p)^2}$$

Or in more general terms when the swimmer is located at (x_s, y_s) ,

$$d_2 = \sqrt{(y_s - W)^2 + (x_s - x_p)^2}$$

Because the optimization is not aimed at the distance the lifeguard has to cover but at the time he takes, each distance has to be divided by the speed of the lifeguard in that respective

medium. The lifeguard has speed v_s in sand and v_w in water. Therefore, the times t_1 and t_2 are equal to:

$$t_1 = \frac{\sqrt{W^2 + x_p^2}}{v_s} \quad (1)$$

$$t_2 = \frac{\sqrt{(y_s - W)^2 + (x_s - x_p)^2}}{v_w} \quad (2)$$

The total time taken is the sum of the two formulae (1) and (2).

$$t_{tot} = t_1 + t_2 = \frac{\sqrt{W^2 + x_p^2}}{v_s} + \frac{\sqrt{(y_s - W)^2 + (x_s - x_p)^2}}{v_w} \quad (3)$$

If this equation is plotted with respect to x_p , there should be a minimum time for $0 \leq x \leq x_s$.

The minimum can be found by differentiation the function and setting its derivative equal to zero. Additionally, it must be determined whether the determined point is actually a minimum. The differentiation is quite simple as it only requires the power and chain rule.

$$\frac{dt}{dx_p} = \frac{x_p}{v_s \sqrt{x_p^2 + W^2}} - \frac{x_s - x_p}{v_w \sqrt{(y_s - W)^2 + (x_s - x_p)^2}}$$

To obtain the minimum, this must equal zero.

$$0 = \frac{x_p}{v_s \sqrt{x_p^2 + W^2}} - \frac{x_s - x_p}{v_w \sqrt{(y_s - W)^2 + (x_s - x_p)^2}}$$

At this point it is recognizable that isolating x_p on one side will be quite the daunting task. One could try to square everything in hopes of opening up the square roots in the denominators. This would require using the binomial formula and it would become quite messy pretty quickly. Trying this reveals a fourth order polynomial that is unsolvable analytically.

Instead, this has to be solved numerically. This can be done using a solver or plotting the function and looking out for the root. For this and all other numerical calculations the Nelder-Mead algorithm was used.

This results in one value that gives the point at which the lifeguard should cross the boundary to reach the swimmer fastest.

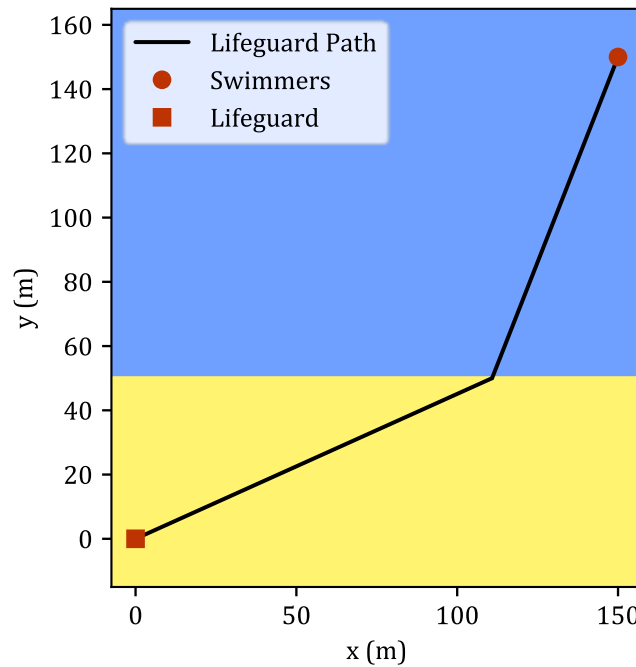


Figure 3: The optimum standard path (own work)

If the lifeguard can run $2\frac{m}{s}$ on land and swim $0.8\frac{m}{s}$ in water, the lifeguard should cross the boundary at point (50 ,110.84) to take a minimum time of 195.04 seconds.

This is a problem that is quite closely linked to the field of optics. In the process of refraction, a light ray travels through the boundary of two media with different densities. The density of a medium is inversely proportional to the wave speed of the light in that medium. Because of this change in wave speed, the light ray bends. The ratio of the sine of the angle of the incident and refracted ray is equal to the ratio of the two wave speeds. Interestingly, this ratio of angles is the case for the optimal path.

2.2 Extension of the Problem

The standard problem is quite simple. Therefore, there are extensions that can be applied to this problem.

1. The shape of the beach is not a straight line but a function.
2. There is a walkway connecting the lifeguard outpost to the water. The lifeguard has the choice when to leave the walkway and when to enter the water. This results in two parameters that can be optimized
3. As in real life, the sand right at the shore is harder and can be run on at a higher speed. Therefore, the final scenario includes an area where the lifeguard can run faster initially

but continuously decreases in speed the closer, he gets to the water. This can be done using a large number of discrete boundaries to simulate a continuous curve.

The aforementioned extensions will be combined in different ways to create optimization problems. In contrary to 2.1, more than one waypoint can be varied to create a time-optimal path. This leads to optimization problems with several variables.

3 How to solve the Problem

The problem to find a path that can be passed in a minimal time under different circumstances is an optimization problem. Therefore, some basics and a solution method is presented.

3.1 Optimization Theory

A scalar optimization problem is determined by a scalar *cost function* $F(z)$ that depends on a tuple z of *input variables* z_i . The size of the tuple is n . Optimizing means to find values for the input variables that bring the cost function either to a maximum or minimum. Usually, it is sufficient to only look at minimization problems, because a maximum search can be converted to a minimum search by changing the sign of the cost function.

For unconstrained optimizations of steady cost functions, the minima can be found at so-called *stationary points*, where the cost function has zero partial derivatives with respect to all input variables. The cost function must be differentiated with respect to each input variable while leaving all others constant.

$$\frac{\partial F}{\partial z_i} = 0 \text{ for } i = 1 \dots n \quad (4)$$

The derivatives lead to n equations that can be solved simultaneously for the n input variables. For one input variable there only exists one equation as shown in section 2.1. Often this solution is not possible analytically. All partial derivatives expressed together as a vector are called the *gradient* of the cost function.

The existence of a stationary point does not guarantee a minimum, because it can also be a maximum or an inflection point (Figure 4) or a saddle point in higher dimensions. Furthermore, a cost function can show several local minima. So further investigations on the character of a stationary point have to be carried out. This normally implies the examination of the second derivative of the cost function what results in the so-called Hesse matrix for optimization problems with more than one input variable. In this investigation, the cost functions did not exhibit stationary points at implausible locations.

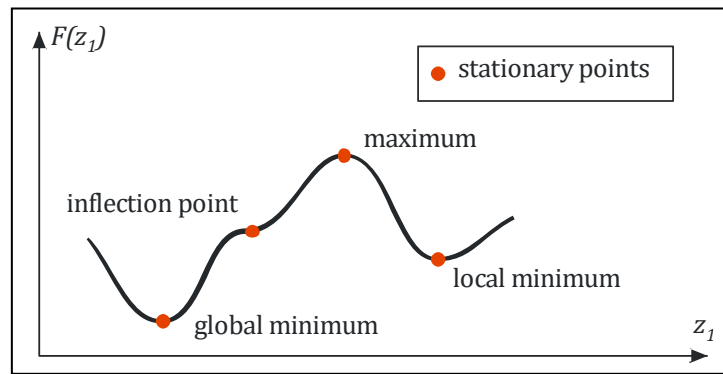


Figure 4: Different types of stationary points of a one-dimensional cost function (own work)

Figure 4 shows the types of stationary points in two dimensions.

Time for Walkway Scenario

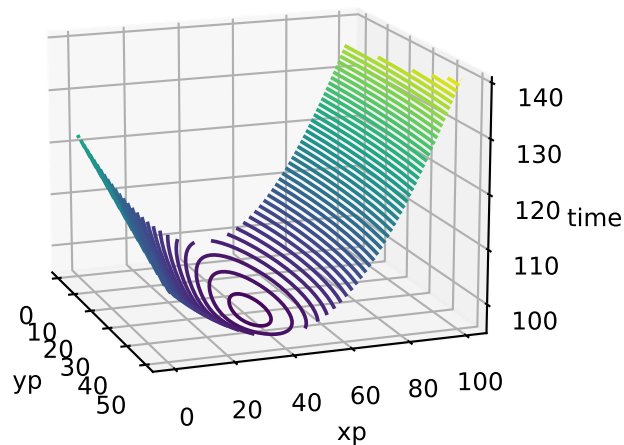


Figure 5: Example of a minimum in two-dimensional optimization (own work)

Since this investigation also deals with two variable optimization, it is also worth mentioning the appearance of a minimum in three dimensional space. This is exhibited in Figure 5.

3.2 Numerical Solution of Optimization Problems

The analytical solution of the equations (4) is often impossible. Also, the problems occurring in this essay shown in section 4 cannot be solved analytically. Therefore, the method of numerical optimization has to be applied. Numerical optimization is one of the deepest investigated problems in numerical mathematics. Decades of research have gained a vast variety of algorithms that all have their individual pros and cons. Most of the algorithms are implemented in numerical libraries available for all commonly used programming languages.

One main distinguishing feature of the different algorithms is the information about the optimization problem they need to be provided. Simple algorithms make only use of the cost function while more elaborated algorithms expect also the gradient or even the Hesse matrix.

After good experiences in the first tests for the problems in this essay, it was decided to use the Nelder-Mead-Algorithm (Nelder and Mead) also known as downhill-simplex-method. It is a quite simple algorithm, that makes use only of the cost function. The usage of the algorithm in the framework of the Python programming language and the SciPy library (SciPy) will be illustrated here. As example for the minimization the two-dimensional “Rosenbrock function” (Rosenbrock) that is commonly used for the test and comparison of different numerical optimization algorithms, will be used.

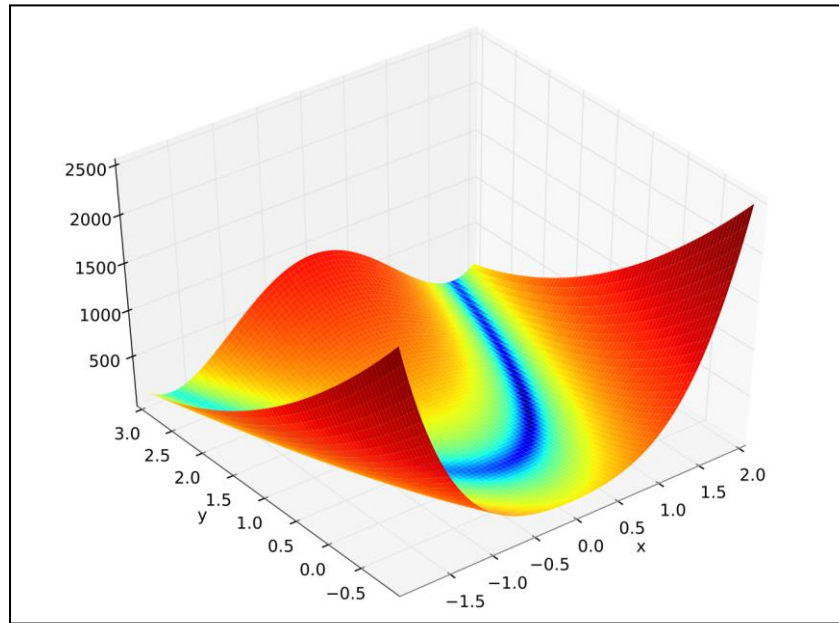


Figure 6: The Rosenbrock function (Gorn). x and y are used instead of z_1 and z_2

The objective function is

$$F(z_1, z_2) = (1 - z_1)^2 + 100(z_2 - z_1^2)^2.$$

The derivatives

$$\frac{\partial F}{\partial z_1} = 2(z_1 - 1) + 400(z_1^3 - z_1 z_2)$$

$$\frac{\partial F}{\partial z_2} = 200(z_2 - z_1^2)$$

can be found easily and it is obvious that $z_1 = 1, z_2 = 1$ is the unique stationary point, based on inspection of the formulae. The Python code that solves the problem is shown here:

```
import numpy as np
from scipy.optimize import minimize

def cf_rosenbrock(z):
    return (1 - z[0])**2 + 100*(z[1] - z[0]**2)**2

z0 = np.array([0, 0])
res = minimize(cf_rosenbrock, z0, method='nelder-mead',
               options={'xatol': 1e-8, 'disp': True})

print(res.x)
```

The cost function is implemented in the subroutine `cf_rosenbrock`. Because of the array numbering conventions of Python the variables z_1 and z_2 appear as `z[0]` and `z[1]`, respectively. As mentioned above, the algorithm needs no further information about the problem. After defining an initial guess of the solution, the solver “minimize” is called. Arguments are the subroutine that provides the cost function, the initial guess, the search algorithm (Nelder-Mead) and some options specifying the tolerances for terminating the search. As result, the algorithm delivers as expected the tuple `[1.0, 1.0]` for z_1 and z_2 at the minimum.

For solving the problems in Section 4 similar Python code is used. Only the cost function is specific for the problems and the number of variables and therefore the size of the initial guess tuple changes. Furthermore, code for generating the result plots is added.

4 Application to different scenarios

4.1 Scenario 1 – Curved Shoreline

A simple way to extend the basic problem is to consider the fact that the waterfront may not be a straight line. In the situation of the straight line, the point at which the lifeguard crosses into the water is defined as (x_p, W) , where W was the distance to the waterfront. Instead, this point could be expressed as $(x_p, f(x_p))$, where f is a function that, unlike W , is in terms of x_p . Because W is technically a function too, it is possible to simply substitute W with $f(x_p)$. Using formula (3), the new total time for the lifeguard is equal to

$$t_{tot} = t_1 + t_2 = \frac{\sqrt{f(x_p)^2 + x_p^2}}{v_s} + \frac{\sqrt{(y_s - f(x_p))^2 + (x_s - x_p)^2}}{v_w}$$

This can once again be differentiated and set equal to zero to get:

$$0 = \frac{x_p}{v_s \sqrt{x_p^2 + f(x_p)^2}} - \frac{x_s - x_p}{v_w \sqrt{(y_s - f(x_p))^2 + (x_s - x_p)^2}}$$

This can again be solved numerically using speeds $v_s = 2 \frac{m}{s}$ and $v_w = 0.8 \frac{m}{s}$.

Given that $f(x) = 5 \sin\left(\frac{x}{15}\right) + 50$, the beach looks as follows.

If a swimmer is added at coordinates (0,100), the fastest path of the lifeguard is not in a straight line. In fact, the lifeguard diverts a little bit to stay on the faster sand for longer, before going into the water.

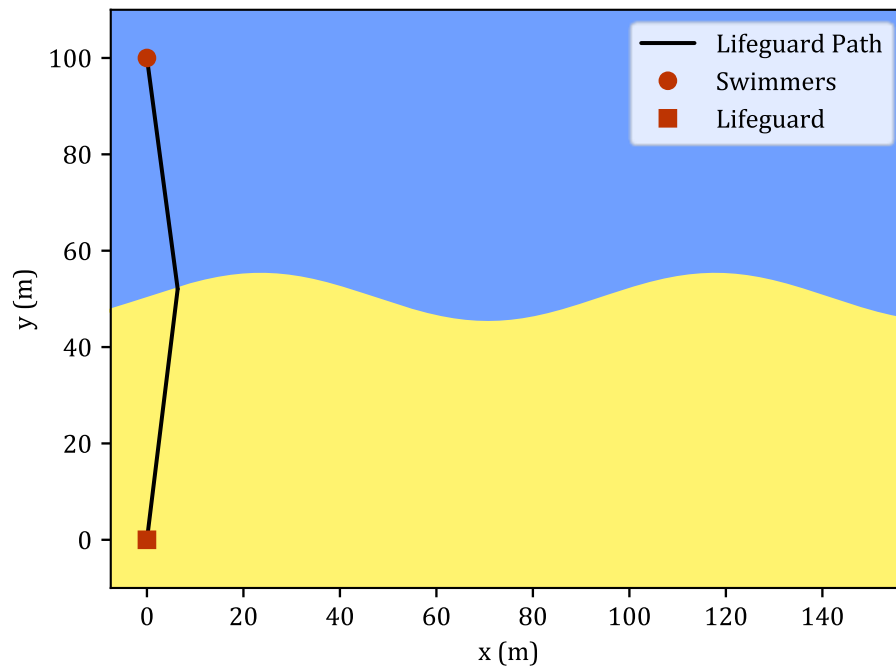


Figure 7: Solution for the optimal path at the beach with non-straight coastline (own work)

Positioning many swimmers at different positions, it becomes obvious that the fastest way always involves entering the water at one of the peaks of the sine function.

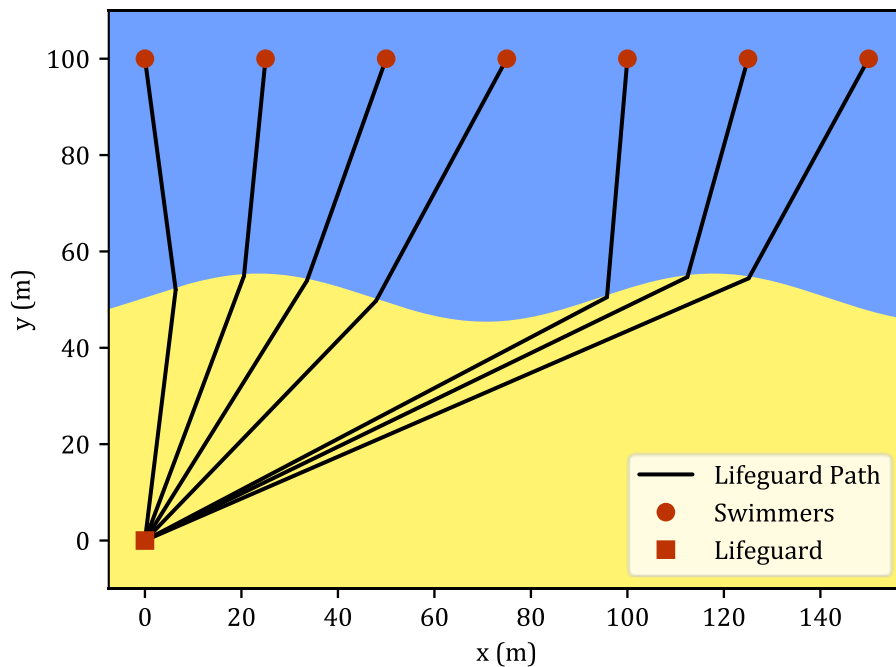


Figure 8: Solutions for the optimal path at the beach with non-straight coastline (own work)

Table 1: Solutions for different swimmer positions for the non-straight coastline

Swimmer (left to right)	Coordinates of swimmer	Coordinates of Water Entry	Time taken for Rescue (seconds)
0	(0,100)	(6.38 ,52.06)	86.68
1	(25,100)	(20.53,54.90)	85.95
2	(50,100)	(33.58,53.93)	92.96
3	(75,100)	(47.90,49.74)	105.90
4	(100,100)	(95.76,50.50)	116.23
5	(125,100)	(112.43,54.68)	121.30
6	(150,100)	(125.13,54.42)	113.13

In summary, this addition to the problem allows for a more accurate depiction of beaches in real life. A sine function was used for this example because many beaches tend to form periodic bumps most closely related to a sine function.

4.2 Scenario 2 – Walkway in the sand

The lifeguard posts at beaches are usually not located right at the waterfront. However, they often have walkways that lead right up to the edge of the water, allowing a lifeguard to quickly enter the water. On this walkway, the lifeguard is able to run much faster with $v_{ww} = 5 \frac{m}{s}$ than on the sand. Figure 9 uses $v_s = 2 \frac{m}{s}$ and $v_w = 0.8 \frac{m}{s}$.

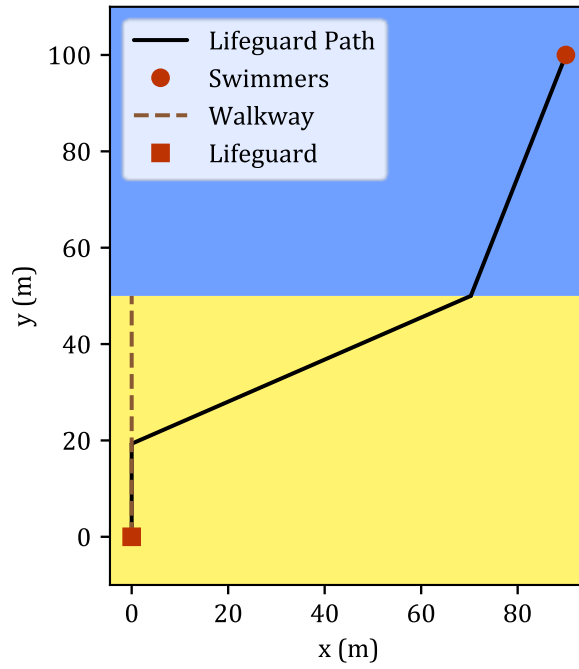


Figure 9: Solution for the optimal path including a walkway (own work)

For this constellation, two parameters can be adjusted. Firstly, when the lifeguard leaves the walkway and when he enters the water. The lifeguard leaves the walkway at point Q with coordinates $(0, y_p)$ and takes a straight path to point P with coordinates (W, x_p) where he enters the water. This splits the path the lifeguard takes into three sections. The time he takes on the walkway, the beach and in water.

The distance the lifeguard covers on the walkway is simply equal to the y-value of the point Q.

$$d_1 = y_p$$

Given walkway speed v_{ww} , the time the lifeguard spends on the path is equal to:

$$t_1 = \frac{y_p}{v_{ww}}$$

The next part of the journey is the connection from point Q to P.

$$d_2 = \sqrt{(W - y_p)^2 + x_p^2}$$

The speed on the sand v_s remains the same in respect to the previous examples.

$$t_2 = \frac{\sqrt{(W - y_p)^2 + x_p^2}}{v_s}$$

The final section is the path through water from point P to the swimmer located at (x_s, y_s) :

$$d_3 = \sqrt{(x_s - x_p)^2 + (y_s - W)^2}$$

Therefore:

$$t_3 = \frac{\sqrt{(x_s - x_p)^2 + (y_s - W)^2}}{v_{water}}$$

The total time in terms of x_p and y_p is therefore:

$$t_{total} = t_1 + t_2 + t_3 = \frac{|y_p|}{v_{walk}} + \frac{\sqrt{(W - y_p)^2 + x_p^2}}{v_s} + \frac{\sqrt{(x_s - x_p)^2 + (y_s - W)^2}}{v_{water}} \quad (5)$$

To solve this analytically, at this point it would be required to differentiate this in terms of x_p and y_p . At this point one would have to solve these equations simultaneously. Since solving something like this analytically is far beyond the scope of an IA, this will be solved numerically. This will once again be done using the Nelder-Mead algorithm. Since this algorithm does not need the gradients of the function to work, the derivative does not need to be formed.

In this case an interesting side-effect can be observed. In equation (5) the absolute value of the way on the walkway has to be used. Otherwise, the optimizing algorithm tries to minimize the total time by running backwards with a negative way y_p gaining negative total time. This is obviously not possible. The cost function (5) is plotted in Figure 10 for swimmer position 2 in Table 2.

Time for Walkway Scenario

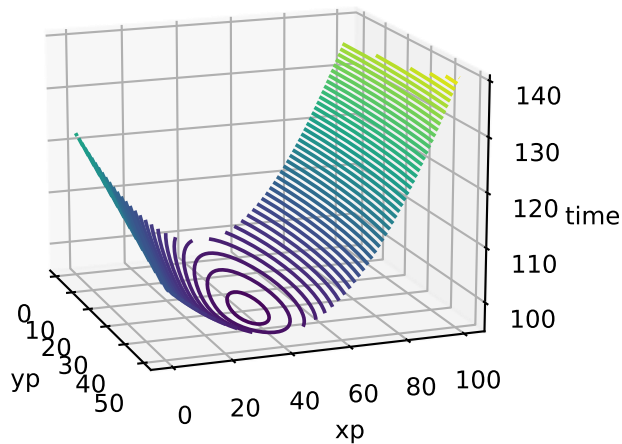


Figure 10: Example cost function for the walkway on the beach scenario (own work)

The graph appears to be similar to a two-dimensional paraboloid with different curvatures in two main directions. The minimum at $y_p \approx 30$ and $x_p \approx 40$ is clearly visible. This is obviously the only stationary point with zero gradient in the whole domain. The graph shows that the variable x_p has a much higher influence on the time the lifeguard takes than the variable y_p . Moving in the y_p direction from the minimum causes the time to increase much less than moving the same distance in the x_p direction.

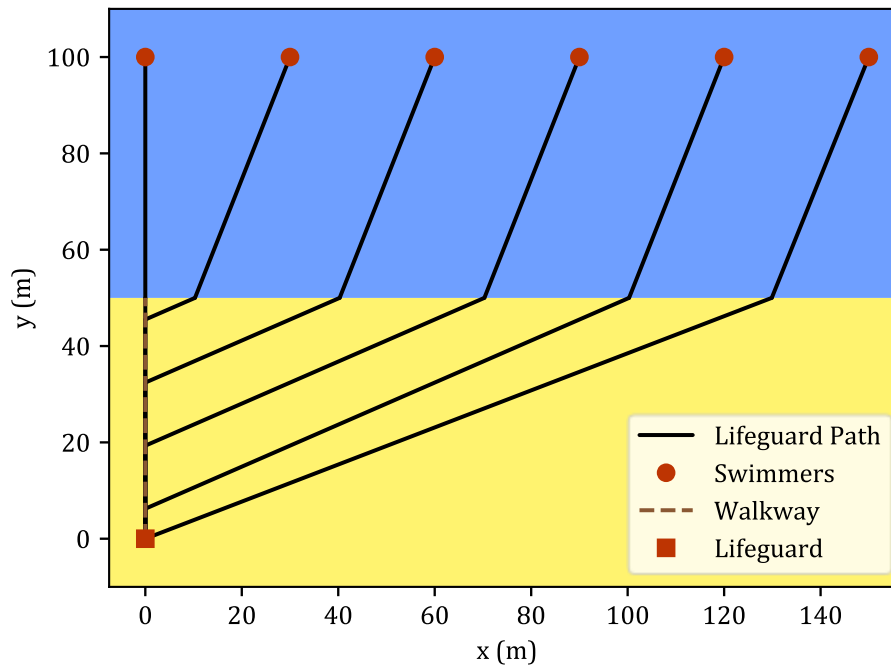


Figure 11: Solutions for the optimal path at the beach with a walkway (own work)

Table 2: Solutions for different swimmer positions at a beach with walkway

Swimmer (left to right)	Coordinates of Swimmer	Coordinates of Walkway Exit	Coordinates of Water Entry	Time taken for Rescue (seconds)
0	(0,100)	(0,50.00)	(0.00,50)	72.50
1	(30,100)	(0,45.51)	(10.30,50)	81.90
2	(60,100)	(0,32.41)	(40.30,50)	95.64
3	(90,100)	(0,19.32)	(70.30,50)	109.39
4	(120,100)	(0,6.23)	(100.30,50)	123.14
5	(150,100)	(0,0.00)	(129.88,50)	136.96

Looking at the graph, it is possible to see the fact that most lines are parallel. This links back to the theory of optics and Snell's law because the angles of the rays are related to the speed. Because the speed of the lifeguard is the same no matter the path, the rays are parallel. This results in an even spacing of the lifeguard leaving the path as well as entering the water for each swimmer. It is worth noting however, that the path for the swimmer at 150 meters is not parallel to the others. This is because for the distance to be the same, the lifeguard would have to leave the path behind the origin. This of course does not work. As mentioned previously, the parameter y_p must always be positive or else the algorithm thinks it is gaining time when moving backwards. In fact, when the absolute value of that variable is not taken, the optimal path from the origin is for the lifeguard to move backwards on the walkway and leave the walkway at the point where the distance to the next path is the same as the others. This is not possible though, because walking backwards does not turn back time.

4.3 Scenario 3 – Speed Gradient

The final extension that can be applied to the problem is a more continuous interpretation of how fast the lifeguard can run on certain areas. The speed in deep water and the speed on land remains the same. For this extension however it is possible to introduce a transition area of length L between sand and deep water. The sand right at the waterfront is saturated with water and therefore more solid and is therefore easier to walk on, allowing for the lifeguard to run more quickly. Additionally, running in shallow water is faster than having to completely swim. Considering this, it is possible to have a linear speed gradient within the transition area that starts off with a faster speed v_{ws} (wet sand) than the dry sand speed and reaches the speed of swimming v_w at the boundary to the deep water. This should quite accurately simulate the effect of compact sand and the possibility to run in shallow water.

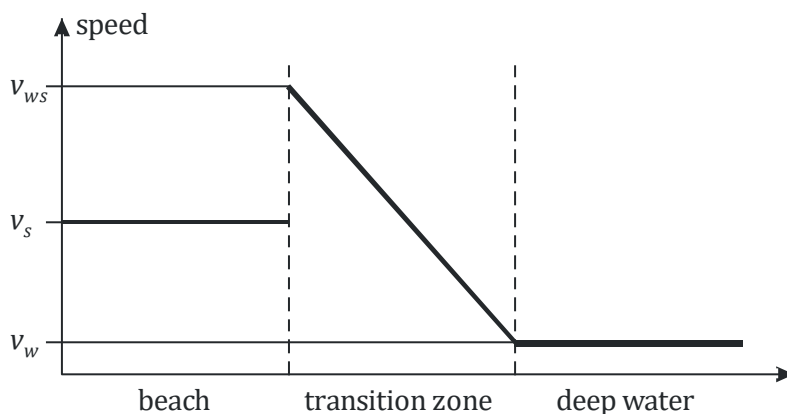


Figure 12: The different speeds in the different sections (own work)

In the formulation with a continuous speed gradient the problem is not solvable as optimization problem with a finite number of input variables. Therefore, the transition zone will be discretized into several zones with constant speed. If these zones are made smaller and smaller, the difference in speed in each one of these zones becomes less and less, making the transition more continuous.

Each zone boundary has one parameter p which dictates the x value of the point at which the lifeguard crosses it. The y value of the point is determined by the number of boundaries and length and position of the transition zone.

If the transition area has n boundaries, the speed in the zone enclosed by the $(n - 1)$ th and n th boundary should be just above the speed in water. Anything beyond the n th boundary should simply equal to v_{water} . These zones must also be classified and numbered. The zeroth zone is enclosed by boundary zero and boundary one. The first zone is enclosed by boundary one and boundary two and so on.

Using these criteria, the speed of the lifeguard in terms of the zone number i is equal to:

$$v(i) = \frac{v_{water} - v_{ws}}{n - 1} \cdot i + v_{ws} \quad (6)$$

Where n is the number of boundaries in the transition zone. When i is equal to zero, the speed $v = v_{ws}$. When i is equal to $n - 1$, corresponding an imaginary zone after the final one in the transition zone, the speed $v = v_{water}$, because this zone represents the water. Using this the overall time inside of the transition time can be determined.

The overall time is dependent on n parameters p_i representing the x coordinates for each boundary. There are $n + 1$ segments that must be summed up to obtain the final time the lifeguard needs. One leading from the lifeguard post to the transition zone, one leading out of the transition zone through the water to the swimmer and $n - 1$ segments within the transition zone. The first segment has the length

$$d_o = \sqrt{W^2 + p_0^2}$$

Using the speed v_s , the time taken for this segment can be calculated.

$$t_0 = \frac{1}{v_s} \sqrt{W^2 + p_0^2}$$

The length of one segment in the transition area can be expressed as

$$d_i = \sqrt{(p_{i+1} - p_i)^2 + \left(\frac{L}{n-1}\right)^2}$$

The parameters p_{i+1} and p_i represent the parameters of the boundaries of enclosing the zone i . L is the width of the transition area.

The time the lifeguard takes to move through the transition area is calculated by summing up the times for crossing all the zones using their respective speed from formula (6).

$$t = \sum_{i=0}^{n-1} \frac{1}{v(i)} \sqrt{(p_{i+1} - p_i)^2 + \left(\frac{L}{n-1}\right)^2}$$

Finally, the lifeguard must leave the transition area and travel to the swimmer through the water.

$$d_{n+1} = \sqrt{(x_s - p_{n-1})^2 + (y_s - (W + L))^2}$$

Therefore, the duration of this part of the journey is equal to:

$$t_{n+1} = \frac{1}{v_w} \sqrt{(x_s - p_{n-1})^2 + (y_s - (W + L))^2}$$

The total time function for this optimization is equal to:

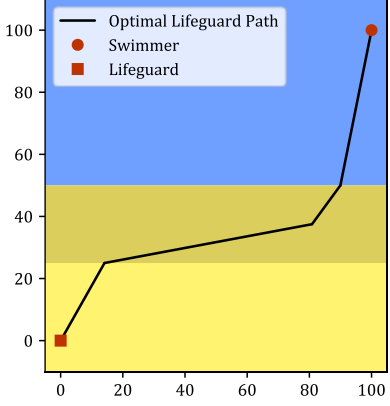
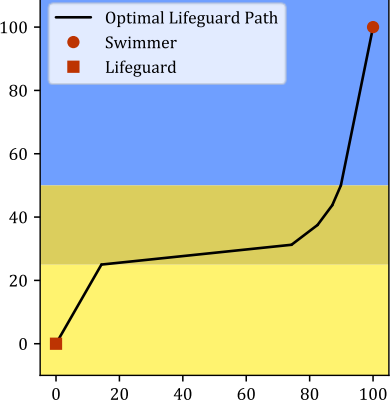
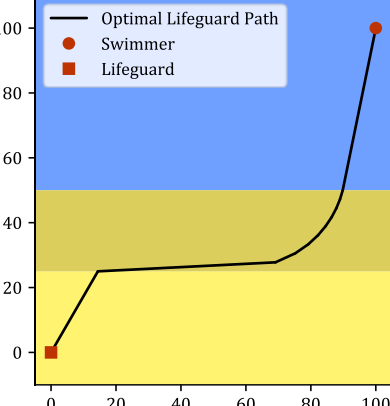
$$\begin{aligned} t_{tot} &= t_0 + t + t_{n+1} \\ &= \frac{1}{v_s} \sqrt{W^2 + p_0^2} + \sum_{i=0}^{n-1} \frac{1}{v(i)} \sqrt{(p_{i+1} - p_i)^2 + \left(\frac{L}{n-1}\right)^2} + \frac{1}{v_w} \sqrt{(x_s - p_{n-1})^2 + (y_s - (W + L))^2} \end{aligned}$$

This function gives the duration of the lifeguard reaching the swimmer in terms of n parameters. This makes an n dimensional optimization problem. Plotting the function and trying to find the minimum through inspection is no longer a viable solution upwards of $n = 2$. The only option that remains here is to use the Nelder-Mead algorithm.

Assume a swimmer at position (100,100). The distance between the lifeguard post and the transition area is 25 m. The transition area is 25 wide. The following are the optimal paths given several values of n . The speeds are $v_s = 2 \frac{m}{s}$, $v_w = 0.8 \frac{m}{s}$ and $v_{ww} = 4$.

Because the fastest the lifeguard can travel is in the beginning of the transfer zone, a large portion of the distance is covered in this region. The slower the lifeguard can travel, the more he orients himself to his target destination.

Table 3: Transition area for various n

$n = 3$	$n = 5$	$n = 10$
		
$t_{tot} = 101.52 \text{ s}$	$t_{tot} = 103.98 \text{ s}$	$t_{tot} = 105.70 \text{ s}$

It is noteworthy that the total time actually increases the more boundaries are added to the problem. This is because the more boundaries actually reduce the distance the lifeguard can run in the fastest zone by making it more realistic.

After $n = 10$, the Nelder-Mead algorithm begins to struggle optimizing all parameters and starts to produce noisy and unrealistic solutions. Optimizing the problem in 10 dimensions gives a very adequate overview of the curve already though.

4.4 Conclusion

Even though some of these scenarios are purely mathematical tinkering, this does have a very real-life application. A trained Lifeguard is able to intuitively judge these things without having to perform multivariable optimization in his head.

This investigation could be extended even more. For example, it can be considered that the lifeguard does not instantly reach the top speed in the medium he is moving in. It would be possible to use kinematics to model the lifeguard accelerating and slowing down when entering the water. This would probably result in an optimum favorizing the beach even more as the lifeguard would accelerate much faster on the beach. It could also be considered that the swimmer is being dragged by currents and is actually moving along the shoreline. That scenario begs the question where the lifeguard could intercept the moving swimmer to save him optimally.

Works Cited

- DLRG. *DLRG Bilanz 2019: 950 Menschen gerettet.* 11 06 2020.
<https://www.dlrg.de/informieren/die-dlrg/presse/dlrg-jahresbilanz/presseinfo-jahresbilanz/>.
- . *DLRG Statistik 2020: mindestens 378 Todesfälle durch Ertrinken.* 09 03 2021.
<https://www.dlrg.de/news/news-detail/dlrg-statistik-2020-mindestens-378-1421-n/>.
- Gorn, Morn the. *Own work, Public Domain.* . n.d.
<https://commons.wikimedia.org/w/index.php?curid=9941741>.
- Nelder, John Asworth and Roger Mead. "A Simplex Method for Function Minimization."
Computer Journal 7 1965: 308–313.
- Rosenbrock, Howard H. "An automatic method for finding the greatest or least value of a function." *The Computer Journal*. 3 (3) 1960: 175–184.
- SciPy. *Homepage.* n.d. <http://www.scipy.org>.
- Tourisme, La Croix Valmer. "Les postes de secours sont ouverts de 10h à 18h." 1 June 2018.